# GitHub Fine-Grained Token - Security & Implementation Guide

## Token Configuration Summary

Token Name: automation-workflow-token

Expiration: March 09, 2026 (90 days)

Created: December 09, 2025

Repository Scope: labgadget015-dotcom/advanced-agent-token-optimizer

Permissions:

- Actions: Read and write
- Contents: Read and write
- Metadata: Read-only (required)

Token Value (copied to clipboard):

github_pat_11BXLoG0QppOJEuPksbLPs_xxAI9T3Ov

## Security Architecture

Principle: Zero Trust, Least Privilege, Defense in Depth

1. Token Storage (Critical Priority)

✔ GitHub Actions Secrets (recommended for CI/CD)

✔ HashiCorp Vault (enterprise)

✔ AWS Secrets Manager / GCP Secret Manager

✔ 1Password CLI / Bitwarden CLI (local development)

✗ NEVER commit to Git repositories

✗ NEVER store in plaintext files

✗ NEVER expose in logs or error messages

2. Scope Limitation

✔ Single repository access only (not all repos)

✔ Read/write only where required (no admin)

✔ 90-day expiration with rotation calendar

✔ Minimal permission set (Actions + Contents only)

3. Rotation Strategy

- Set calendar reminder: February 28, 2026
- Generate new token 1 week before expiration
- Update all deployment targets
- Revoke old token after verification
- Document rotation in audit log

GitHub Actions Implementation

Step 1: Add Token to Repository Secrets

1. Navigate to: https://github.com/labgadget015-dotcom/advanced-agent-token-optimizer/settings/secrets/actions

2. Click "New repository secret"

3. Name: AUTOMATION_WORKFLOW_TOKEN

4. Value: [paste token from clipboard]

5. Click "Add secret"

Step 2: Workflow Configuration

Create: .github/workflows/ci-automation.yml

```yaml
name: AI Automation CI/CD

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

jobs:
  automation-workflow:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout repository
      uses: actions/checkout@v4
```

```
      with:
        token: ${{ secrets.AUTOMATION_WORKFLOW_TOKEN
}}


    - name: Setup Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'


    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt


    - name: Run AI agent tests
      env:
        GITHUB_TOKEN: ${{
secrets.AUTOMATION_WORKFLOW_TOKEN }}
      run: |
```

```
        pytest tests/ --verbose



    - name: Deploy to staging

      if: github.ref == 'refs/heads/develop'

      env:

        GITHUB_TOKEN: ${{
secrets.AUTOMATION_WORKFLOW_TOKEN }}

      run: |

        ./scripts/deploy-staging.sh
```

CLI Usage Examples

Local Development Setup

```
# Option 1: Environment Variable (session-only)

export
GITHUB_TOKEN="github_pat_11BXLoG0QppOJEuPksbLPs_x
xAI9T3Ov"
```

# Option 2: .env file (DO NOT COMMIT)

echo 'GITHUB_TOKEN=github_pat_11BXLoG0QppOJEuPksbLPs_xxAI9T3Ov' >> .env

echo '.env' >> .gitignore

# Option 3: 1Password CLI (recommended)

op item get "GitHub Automation Token" --fields token

export GITHUB_TOKEN=$(op item get "GitHub Automation Token" --fields token)

GitHub CLI (gh) Examples

# Authenticate

gh auth login --with-token < token.txt

# Repository operations

gh repo view labgadget015-dotcom/advanced-agent-token-optimizer

```
gh repo clone
labgadget015-dotcom/advanced-agent-token-optimizer


# Workflow operations

gh workflow list

gh workflow run ci-automation.yml

gh run list --workflow=ci-automation.yml


# API calls

gh api
/repos/labgadget015-dotcom/advanced-agent-token-optimizer/a
ctions/workflows

gh api
/repos/labgadget015-dotcom/advanced-agent-token-optimizer/c
ontents/README.md
```

Python SDK Example

```
from github import Github

import os
```

```
# Initialize with token

g = Github(os.environ['GITHUB_TOKEN'])



# Get repository

repo =
g.get_repo('labgadget015-dotcom/advanced-agent-token-optimi
zer')



# List workflows

for workflow in repo.get_workflows():

    print(f'{workflow.name}: {workflow.state}')



# Trigger workflow

workflow = repo.get_workflow('ci-automation.yml')

workflow.create_dispatch(ref='main')



# Read file contents

contents = repo.get_contents('README.md')
```

```
print(contents.decoded_content.decode())
```

## Security Monitoring & Audit

### Token Activity Monitoring

1. Access audit log:

   https://github.com/settings/security-log

2. Filter for token usage:

   • Search: "automation-workflow-token"

   • Look for: oauth_access.create, oauth_access.use

   • Monitor failed authentication attempts

   • Check for unexpected IP addresses or locations

3. Set up alerts:

   • Enable GitHub security advisories

   • Configure email notifications for:

- New token usage from unknown locations

- Failed authentication attempts

- Permission escalation requests

4. Regular audit checklist (weekly):

☑ Review token activity in security log

☑ Verify no unauthorized access attempts

☑ Check workflow run logs for anomalies

☑ Confirm token still needed with current permissions

☑ Validate expiration date is tracked

5. Incident response plan:

• If token compromised:

1. Immediately revoke at: https://github.com/settings/personal-access-tokens

2. Generate new token with different name

3. Update all deployment targets

4. Review security log for unauthorized actions

5. Document incident and remediation steps

## Compliance & Documentation

1. Token inventory spreadsheet:
   - Token name
   - Creation date
   - Expiration date
   - Repository scope
   - Permissions granted
   - Owner/team responsible
   - Rotation history

2. Change log:
   - 2025-12-09: Created automation-workflow-token
     - Scope: advanced-agent-token-optimizer
     - Permissions: Actions (R/W), Contents (R/W)
     - Expiration: 2026-03-09

- Owner: DevOps Team

Advanced Configuration

Multi-Agent Orchestration Setup

For AI agent workflows requiring GitHub API access:

```
# Python orchestration example
import asyncio
from github import Github
import os

class GitHubAgentOrchestrator:
    def __init__(self):
        self.client = Github(os.environ['GITHUB_TOKEN'])
        self.repo = self.client.get_repo(
            'labgadget015-dotcom/advanced-agent-token-optimizer'
```

```
        )

    async def trigger_workflow(self, workflow_name,
inputs=None):
        """Trigger GitHub Actions workflow with parameters"""
        workflow = self.repo.get_workflow(workflow_name)
        workflow.create_dispatch(
            ref='main',
            inputs=inputs or {}
        )
        return f"Triggered {workflow_name}"

    async def monitor_runs(self, workflow_name):
        """Monitor workflow execution status"""
        runs = self.repo.get_workflow_runs(
            workflow_name=workflow_name
        )
        for run in runs[:5]:  # Latest 5 runs
```

```
            print(f"{run.id}: {run.status} - {run.conclusion}")


    async def read_artifacts(self, run_id):
        """Download and process workflow artifacts"""
        run = self.repo.get_workflow_run(run_id)
        for artifact in run.get_artifacts():
            print(f"Artifact: {artifact.name} ({artifact.size_in_bytes}
bytes)")
```

Token Rotation Automation

```
# Bash script for automated rotation
#!/bin/bash
# rotate-github-token.sh

OLD_TOKEN_NAME="automation-workflow-token"
NEW_TOKEN_NAME="automation-workflow-token-$(date
+%Y%m%d)"
EXPIRATION_DAYS=90
```

```
echo "Starting token rotation process..."

# Step 1: Generate new token (manual step - requires web UI)
echo "1. Generate new token via GitHub UI"
echo "2. Copy token value"
read -sp "Paste new token: " NEW_TOKEN
echo

# Step 2: Update GitHub Actions secrets
gh secret set AUTOMATION_WORKFLOW_TOKEN --body "$NEW_TOKEN" \
  --repo labgadget015-dotcom/advanced-agent-token-optimizer

# Step 3: Update local environment
op item edit "GitHub Automation Token" token="$NEW_TOKEN"

# Step 4: Verify new token works
```

```
GITHUB_TOKEN="$NEW_TOKEN" gh repo view
labgadget015-dotcom/advanced-agent-token-optimizer



if [ $? -eq 0 ]; then

    echo "✓ New token verified successfully"

    echo "✓ Old token can now be revoked manually"

    echo "Next rotation due: $(date -d "+$EXPIRATION_DAYS
days" +%Y-%m-%d)"

else

    echo "✗ Token verification failed - DO NOT revoke old
token"

    exit 1

fi



HashiCorp Vault Integration



# Store token in Vault

vault kv put secret/github/automation-token \

  token="github_pat_11BXLoG0QppOJEuPksbLPs_xxAI9T3Ov"
\
```

```
  expiration="2026-03-09" \
  repository="advanced-agent-token-optimizer"

# Retrieve token in application
vault kv get -field=token secret/github/automation-token

# GitHub Actions with Vault
- name: Get GitHub token from Vault
  uses: hashicorp/vault-action@v2
  with:
    url: ${{ secrets.VAULT_ADDR }}
    token: ${{ secrets.VAULT_TOKEN }}
    secrets: |
      secret/github/automation-token token | GITHUB_TOKEN
```

## Troubleshooting Common Issues

Issue: "Bad credentials" or 401 Unauthorized

Causes:

• Token expired (check expiration date: 2026-03-09)
• Token revoked
• Incorrect token format (missing github_pat_ prefix)
• Token not properly exported to environment

Solutions:
1. Verify token format: echo $GITHUB_TOKEN | grep "github_pat_"
2. Check token status: https://github.com/settings/personal-access-tokens
3. Re-export token: export GITHUB_TOKEN="github_pat_..."
4. Test with: gh auth status

Issue: "Resource not accessible by personal access token"

Causes:
• Insufficient permissions for requested operation
• Trying to access repository outside scope
• Organization settings block fine-grained tokens

Solutions:
1. Verify current permissions: Check token configuration
2. Add required permission via token settings
3. Confirm repository in scope: advanced-agent-token-optimizer only
4. Check org policy: Settings → Third-party access

Issue: GitHub Actions workflow fails with token

Causes:
• Secret not configured in repository
• Secret name mismatch
• Workflow doesn't have secret access

Solutions:
1. Verify secret exists:
   gh secret list --repo labgadget015-dotcom/advanced-agent-token-optimizer
2. Check secret name matches: AUTOMATION_WORKFLOW_TOKEN
3. Ensure workflow references correct secret:
   ${{ secrets.AUTOMATION_WORKFLOW_TOKEN }}
4. Re-add secret if needed

Issue: Rate limiting

Causes:
• Exceeded GitHub API rate limits
• Multiple concurrent requests

Solutions:
1. Check rate limit status:
   gh api rate_limit

2. Implement exponential backoff
3. Cache responses when possible
4. Use conditional requests (ETags)

Debugging Commands

```
# Test token validity
curl -H "Authorization: token $GITHUB_TOKEN" \
  https://api.github.com/user

# Check token scopes
curl -I -H "Authorization: token $GITHUB_TOKEN" \
  https://api.github.com/user | grep x-oauth-scopes

# Verify repository access
gh api /repos/labgadget015-dotcom/advanced-agent-token-optimizer

# Test workflow trigger
gh workflow run ci-automation.yml --repo \
  labgadget015-dotcom/advanced-agent-token-optimizer
```